Lab 7
Sequential Testbench & Two-bit Adder

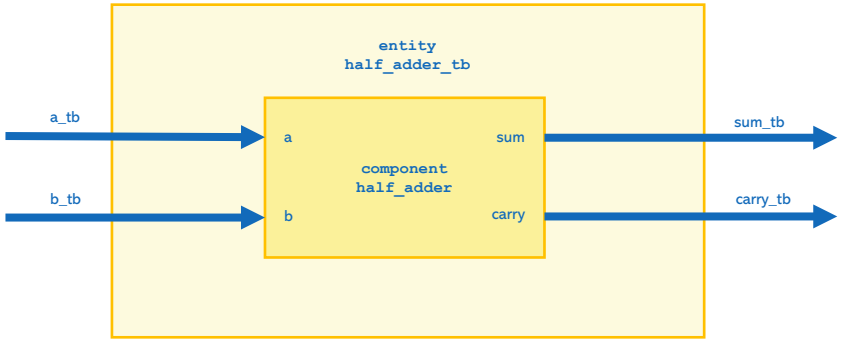
Digital System Fundamentals

Sorayut Glomglome

1. SequentialTestbench

Last Lab Testbench | Concurrent

```
21
22  library IEEE;
23  use IEEE.STD_LOGIC_1164.ALL;
24
33
34  entity ha_tb is
35  --  Port ( );
36  end ha_tb;
37
38  architecture Behavioral of ha_tb is
39
40  component ha is port(
41      a : in STD_LOGIC;
42      b : in STD_LOGIC;
43      sum : out STD_LOGIC;
44      carry : out STD_LOGIC);
45  end component;
46
47  signal a_tb, b_tb, sum_tb, carry_tb : std_logic;
48
49  begin
50
51      uut : ha port map (a => a_tb,
52                         b => b_tb,
53                         sum => sum_tb,
54                         carry => carry_tb);
55
56      a_tb <= '0', '1' after 40 ns;
57      b_tb <= '0', '1' after 20 ns, '0' after 40 ns, '1' after 60 ns;
58
59  end Behavioral;
60
```



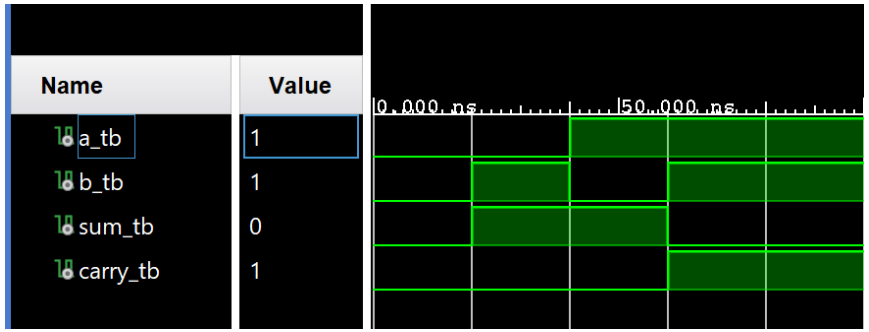
- Concurrent => Difficult
- Manual output checking

Sequential Testbench

1. Simple testbench with concurrent statements.

2. Use **process, assertion, and report** statements.

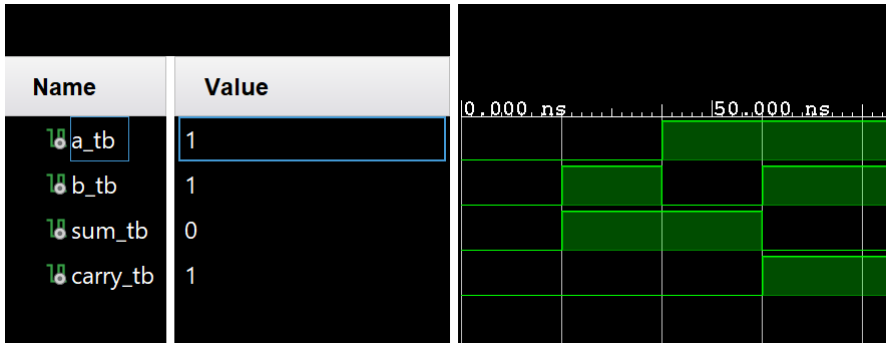
   - Verify output automatically.
   - Print error message.



1.1 Process, Assertion, and Report Statements

Process & Wait Statement

- Sequential execution
- Similar to other high level languages.
- wait for / wait




```
48
49  begin
50
51      uut : half_adder port map (a => a_tb,
52                                 b => b_tb,
53                                 sum => sum_tb,
54                                 carry => carry_tb);
55
56    tb : process
57
58          constant period: time := 20 ns;
59
60          begin
61
62              -- case 0
63              -- input combination a, b ==> 0, 0
64              -- expected output carry, sum ==> 0, 0
65              a_tb <= '0';
66              b_tb <= '0';
67              wait for period;
71
72              -- case 1
73              -- input combination a, b ==> 0, 1
74              -- expected output carry, sum ==> 0, 1
75              a_tb <= '0';
76              b_tb <= '1';
77              wait for period;
81
82              -- case 2
83              -- input combination a, b  ==> 1, 0
84              -- expected output carry, sum ==> 0, 1
85              a_tb <= '1';
86              b_tb <= '0';
87              wait for period;
91
92              -- case 3
93              -- input combination a, b  ==> 1, 1
94              -- expected output carry, sum ==> 1, 0
95              a_tb <= '1';
96              b_tb <= '1';
97              wait for period;
101
102             wait; -- wait indefinitely aka suspend process
103         end process;
104
105 end Behavioral;
```

Assertion & Report Statements

- Check output automatically.
- Print error message onto Tcl console.

```
-- case 0
-- input combination a, b ==> 0, 0
-- expected output carry, sum ==> 0, 0
a_tb <= '0';
b_tb <= '0';
wait for period;
assert (( carry_tb = '0') and (sum_tb = '0'))
report "===> Test failed for input combination 00 <===="
severity error;
```

```
Tcl Console   Messages   Log
# }
# run 1000ns
Error: ===> Test failed for input combination 00 <===
Time: 20 ns  Iteration: 0  Process: /half_adder_tb2/tb  File: G:/Xi
INFO: [USF-XSim-96] XSim completed. Design snapshot 'half_adder_tb2
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:04 ; elapsed = 00:00:08 .
```

```
-- case 0
-- input combination a, b ==> 0, 0
-- expected output carry, sum ==> 0, 0
a_tb <= '0';
b_tb <= '0';
wait for period;
assert (( carry_tb = '0') and (sum_tb = '1'))
report "===> Test failed for input combination 00 <===="
severity error;
```

Complete Code

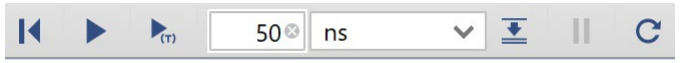
- Complete code for process, assertion and report statements.

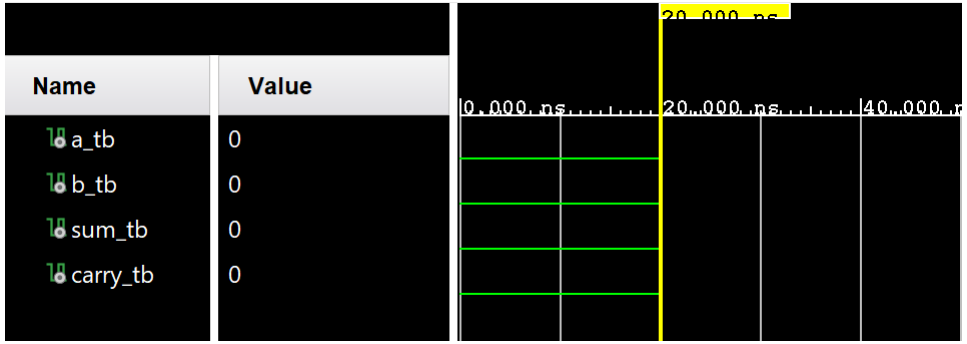
```
tb : process

    constant period: time := 20 ns;

    begin

        -- case 0
        -- input combination a, b ==> 0, 0
        -- expected output carry, sum ==> 0, 0
        a_tb <= '0';
        b_tb <= '0';
        wait for period;
        assert (( carry_tb = '0') and (sum_tb = '0'))
        report "===> Test failed for input combination 00 <===="
        severity error;

        -- case 1
        -- input combination a, b ==> 0, 1
        -- expected output carry, sum ==> 0, 1
        a_tb <= '0';
        b_tb <= '1';
        wait for period;
        assert (( carry_tb = '0') and (sum_tb = '1'))
        report "===> Test failed for input combination 01 <===="
        severity error;

        -- case 2
        -- input combination a, b  ==> 1, 0
        -- expected output carry, sum ==> 0, 1
        a_tb <= '1';
        b_tb <= '0';
        wait for period;
        assert (( carry_tb = '0') and (sum_tb = '1'))
        report "===> Test failed for input combination 10 <===="
        severity error;

        -- case 3
        -- input combination a, b  ==> 1, 1
        -- expected output carry, sum ==> 1, 0
        a_tb <= '1';
        b_tb <= '1';
        wait for period;
        assert (( carry_tb = '1') and (sum_tb = '0'))
        report "===> Test failed for input combination 11 <===="
        severity error;

        wait; -- wait indefinitely aka suspend process
    end process;
```

Breakpoint

```
56  tb : process
57
58      constant period: time := 20 ns;
59
60      begin
61
62          -- case 0
63          -- input combination a, b ==> 0, 0
64          -- expected output carry, sum ==> 0, 0
65          a_tb <= '0';
66          b_tb <= '0';
67          wait for period;
68          assert (( carry_tb = '0') and (sum_tb = '0'))
69          report "===> Test failed for input combination 00 <===="
70          severity error;
71
72          -- case 1
73          -- input combination a, b ==> 0, 1
74          -- expected output carry, sum ==> 0, 1
75          a_tb <= '0';
76          b_tb <= '1';
77          wait for period;
78          assert (( carry_tb = '0') and (sum_tb = '1'))
79          report "===> Test failed for input combination 01 <===="
80          severity error;
81
```

- Restart
- Run All
- Run for 50 ns
- Step
- Break
- Relaunch Simulation





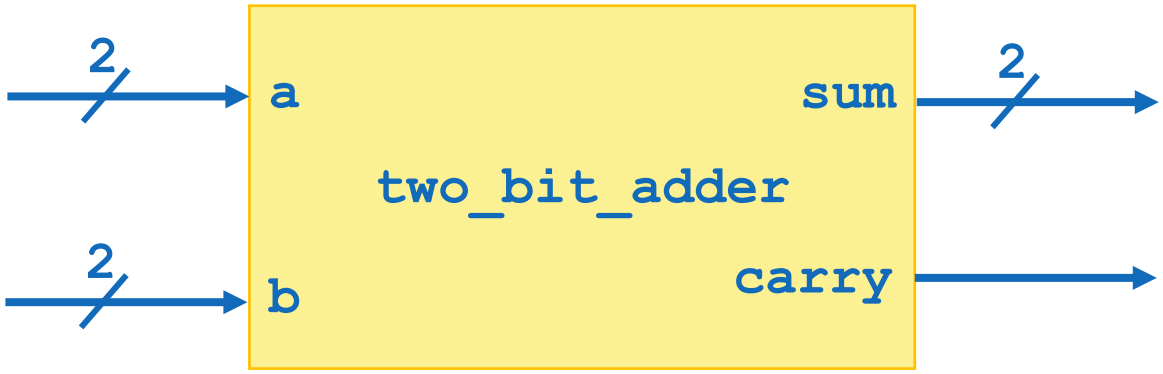
1.2 Checkpoint 1

Checkpoint 1

- Test half adder from Lab 6 using the sequential testbench and provide *one failed case*.

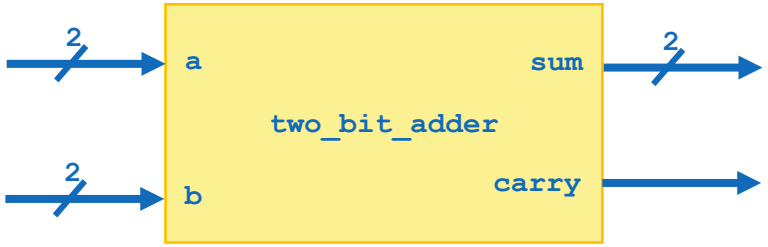
2. Two-bit Adder

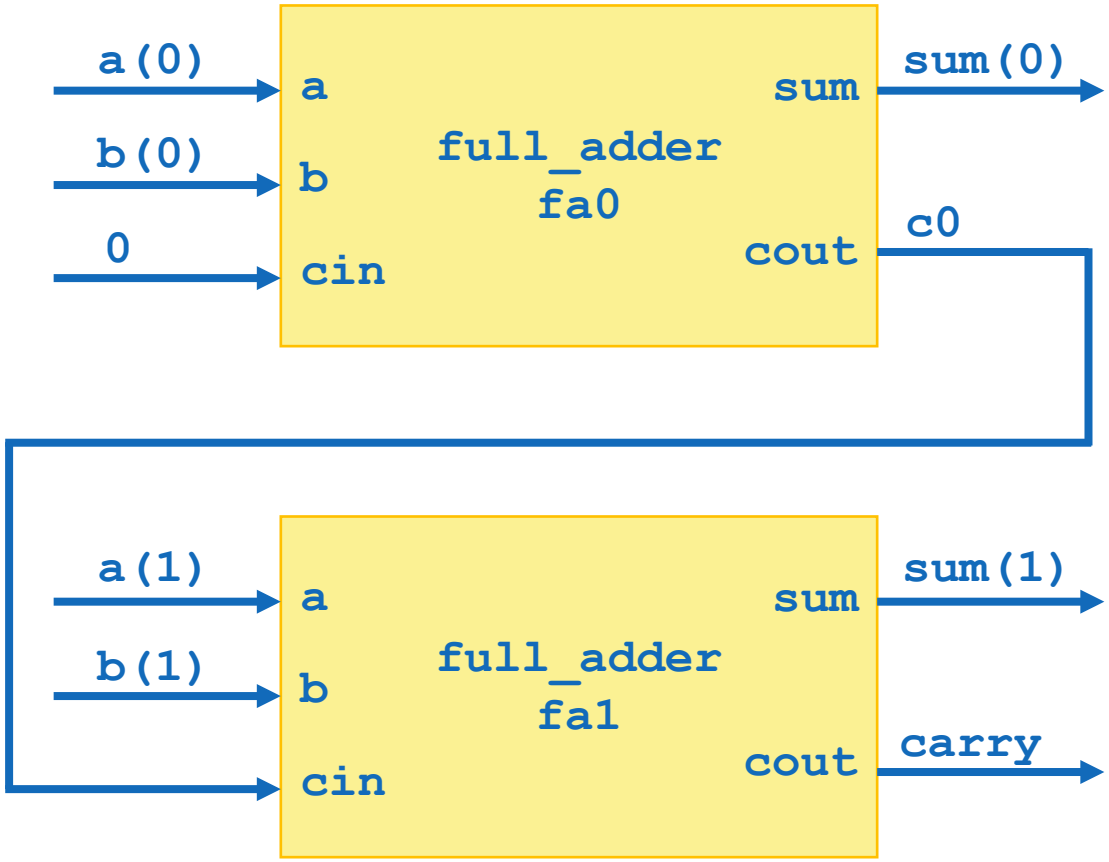
Entity | Two-Bit Adder




```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity two_bit_adder is
    port ( a : in STD_LOGIC_VECTOR (1 downto 0);
           b : in STD_LOGIC_VECTOR (1 downto 0);
           sum : out STD_LOGIC_VECTOR (1 downto 0);
           carry : out STD_LOGIC);
end two_bit_adder;
```

Component | Full Adder






```
41 architecture dataflow of two_bit_adder is
42
43     component full_adder is
44         port ( a, b, cin : in STD_LOGIC;
45                sum, cout : out STD_LOGIC);
46     end component;
47
48     signal c0 : std_logic;
49
50 begin
51
52     fa0: full_adder port map (a=>a(0), b=>b(0), cin=>'0', sum=>sum(0), cout=>c0);
53     fa1: full_adder port map (a=>a(1), b=>b(1), cin=>c0, sum=>sum(1), cout=>carry);
54
55
56 end dataflow;
```

Step

1. Add the *first* design source file for full adder entity.
   a) Use VHDL code from Lab 6.
2. Add the *second* design source file for two-bit adder.
   a) Declare full adder component using the same port names from the first file.
   b) Port mapping two full adders.

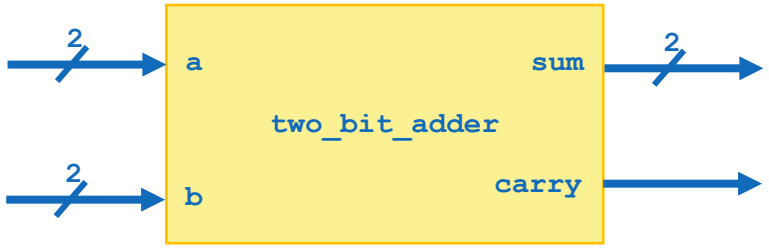
2.1 Checkpoint 2

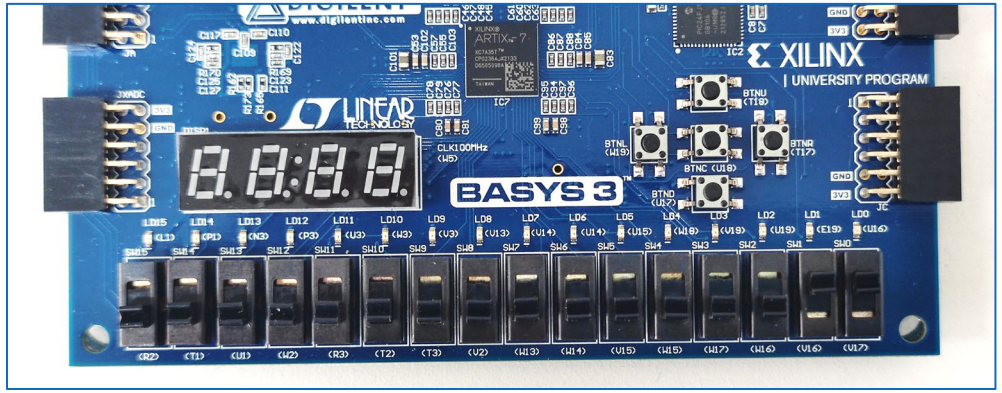
Checkpoint 2

1. Construct two-bit adder from full adders and do the sequential testbench with all possible input cases and provide *two failed cases*.

Checkpoint 2

2. Perform hardware test on FPGA using the following constraints.





Port	I/O Type	Label	Pin
a(1)	switch	SW3	W17
a(0)	switch	SW2	W16
b(1)	switch	SW1	V16
b(0)	switch	SW0	V17
sum(1)	LED	LD1	E19
sum(0)	LED	LD0	U16
carry	LED	LD2	U19

Testbench for two_bit_adder

```
21
22  library IEEE;
23  use IEEE.STD_LOGIC_1164.ALL;
24
25  -- Uncomment the following library declaration if using...
33
34  entity  two_bit_adder_tb is
35  --  Port ( );
36  end two_bit_adder_tb;
37
38  architecture behavior of two_bit_adder_tb is
39
40      -- 1. Declare the Unit Under Test (UUT)
41      component two_bit_adder
42          port (
43              a    : in  std_logic_vector(1 downto 0);
44              b    : in  std_logic_vector(1 downto 0);
45              cin  : in  std_logic;
46              sum  : out std_logic_vector(1 downto 0);
47              cout : out std_logic
48          );
49      end component;
50
51      -- 2. Define the Record for Test Patterns
52      type test_pattern_t is record
53          a_in     : std_logic_vector(1 downto 0);
54          b_in     : std_logic_vector(1 downto 0);
55          cin      : std_logic;
56          exp_sum  : std_logic_vector(1 downto 0);
57          exp_cout : std_logic;
58      end record;
59
60      -- 3. Define an Array of these Records (The Test Suite)
61      type test_array_t is array (natural range <>) of test_pattern_t;
62      constant test_table : test_array_t := (
63          -- (A, B, Cin, Sum_Expected, Cout_Expected)
64          (a_in => "00", b_in => "00", cin => '0', exp_sum => "00", exp_cout => '0'), -- 0+0+0=0
65          (a_in => "01", b_in => "01", cin => '0', exp_sum => "10", exp_cout => '0'), -- 1+1+0=2
66          (a_in => "10", b_in => "10", cin => '0', exp_sum => "00", exp_cout => '1'), -- 2+2+0=4
67          (a_in => "11", b_in => "11", cin => '1', exp_sum => "11", exp_cout => '1')  -- 3+3+1=7
68      );
69
```

```
69
70      -- 4. Local Signals to connect to UUT
71      signal a_tb, b_tb    : std_logic_vector(1 downto 0);
72      signal cin_tb     : std_logic;
73      signal sum_tb     : std_logic_vector(1 downto 0);
74      signal cout_tb    : std_logic;
75
76  begin
77
78      -- Instantiate the UUT
79      uut: two_bit_adder
80          port map (
81              a => a_tb, b => b_tb, cin => cin_tb, sum => sum_tb, cout => cout_tb
82          );
83
84      -- Stimulus Process
85      stim_proc: process
86
87      constant period: time := 10 ns;
88
89      begin
90
91          --a_tb(1) <= '1';
92          --a_tb(0) <= '0';
93          a_tb <= "10";
94
95          b_tb(1) <= '1';
96          b_tb(0) <= '0';
97
98          cin_tb <= '0';
99
100         wait for period;
101
102         assert ( sum_tb = "00" and cout_tb = '1')
103             report "=====>> Test Error <<=====" 
104             severity error;
105
```

```
106         for i in test_table'range loop
107             -- Apply inputs from the record
108             a_tb   <= test_table(i).a_in;
109             b_tb   <= test_table(i).b_in;
110             cin_tb <= test_table(i).cin;
111
112             wait for period; -- Allow signals to propagate
113
114             -- Verify outputs against expected record values
115             assert (sum_tb = test_table(i).exp_sum and cout_tb = test_table(i).exp_cout)
116                 report "Test Case " & integer'image(i) & " failed!"
117                 severity error;
118         end loop;
119
120         report "=====>>>>> Simulation Finished Successfully! <<<<<=====";
121         wait; -- Stop the process
122
123     end process;
124
125 end behavior;
126
```

Record, Array & Loop

```
    -- 2. Define the Record for Test Patterns
    type test_pattern_t is record
        a_in     : std_logic_vector(1 downto 0);
        b_in     : std_logic_vector(1 downto 0);
        cin      : std_logic;
        exp_sum  : std_logic_vector(1 downto 0);
        exp_cout : std_logic;
    end record;

    -- 3. Define an Array of these Records (The Test Suite)
    type test_array_t is array (natural range <>) of test_pattern_t;
    constant test_table : test_array_t := (
        -- (A, B, Cin, Sum_Expected, Cout_Expected)
        (a_in => "00", b_in => "00", cin => '0', exp_sum => "00", exp_cout => '0'), -- 0+0+0=0
        (a_in => "01", b_in => "01", cin => '0', exp_sum => "10", exp_cout => '0'), -- 1+1+0=2
        (a_in => "10", b_in => "10", cin => '0', exp_sum => "00", exp_cout => '1'), -- 2+2+0=4
        (a_in => "11", b_in => "11", cin => '1', exp_sum => "11", exp_cout => '1')  -- 3+3+1=7
    );

```

```
        for i in test_table'range loop
            -- Apply inputs from the record
            a_tb   <= test_table(i).a_in;
            b_tb   <= test_table(i).b_in;
            cin_tb <= test_table(i).cin;

            wait for period; -- Allow signals to propagate

            -- Verify outputs against expected record values
            assert (sum_tb = test_table(i).exp_sum and cout_tb = test_table(i).exp_cout)
                report "Test Case " & integer'image(i) & " failed!"
                severity error;
        end loop;

```

This section of the testbench uses a loop to automate testing by pulling data from the **array of records** you defined earlier.

1. The Loop: `for i in test_table'range loop`

- `i` : This is a loop iterator (an integer) that automatically exists only within this loop.
- `test_table'range` : This uses a **VHDL Attribute** (`'range`). Instead of hardcoding `0 to 3`, `'range` tells the compiler to look at the `test_table` array and automatically use its defined indices.
  - *Benefit:* If you add more test cases to the table later, you don't need to change this line; the loop will automatically cover the new entries.

2. Assignment: `a <= test_table(i).a_in;`

- `test_table(i)` : This accesses the *i*-th "row" (record) in your array.
- `.a_in` : This is the **Dot Notation**. It specifies exactly which field inside that record you want to access.
- **Result**: The value stored in the record for that specific test case is driven onto the signal `a`, which is connected to the adder's input.

3. Timing: `wait for 10 ns;`

- In VHDL simulation, signal assignments (`<=`) do not happen instantly; they are scheduled for the next "delta cycle."
- This line suspends the process for 10 nanoseconds, allowing the "logic" of your Full Adders to process the inputs and update the `sum` and `cout` signals before you check them.

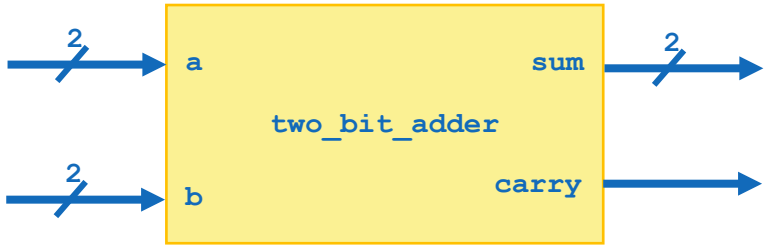
4. Verification: `assert (...) report ... severity ...;`

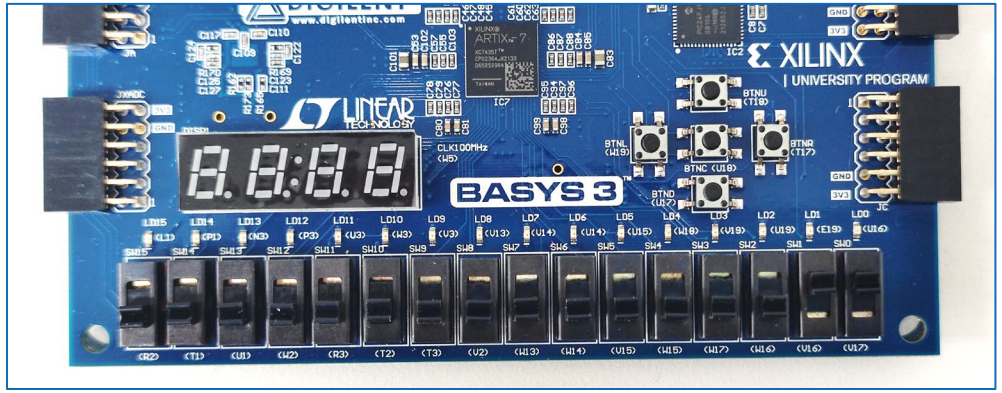
This is a built-in VHDL debugging tool:

- `assert (sum = test_table(i).exp_sum ...)` : The simulator checks if this condition is **TRUE**.
- `report "..."` : If the condition is **FALSE**, the simulator prints this specific message to the console.
  - `integer'image(i)` : This converts the current loop index `i` into a string so you know exactly which test case failed.
- `severity error` : This tells the simulator how serious the failure is. `error` usually flags the simulation log, while `failure` can be used to stop the simulation entirely.

Checkpoint 2

2. Perform hardware test on FPGA using the following constraints.





Port	I/O Type	Label	Pin
a(1)	switch	SW3	W17
a(0)	switch	SW2	W16
b(1)	switch	SW1	V16
b(0)	switch	SW0	V17
sum(1)	LED	LD1	E19
sum(0)	LED	LD0	U16
carry	LED	LD2	U19

Lab 7 Submission

Date _____ **Group No.** _____

Questions

1. Implement 4-bit adder from 4 full adders using VHDL.

2. Write Gen AI prompt used to create **test_table** with all input scenarios for 4-bit adder from Question 1.

THANK YOU